

Séance 1 — Introduction à Matlab/Octave

Rappel théorique

Pour MATLAB (dont le nom vient de *MATrix LABoratory*), l'élément central pour les opérations est la matrice (ce qui comporte les vecteurs comme cas particuliers). La déclaration d'un vecteur se fait via les crochets :

```
x = [x1, x2, ...]
```

Les virgules pour séparer les éléments sont facultatives. L'instruction suivante est donc équivalente à la précédente :

```
x = [x1 x2 ...]
```

Une matrice se déclare comme une suite de vecteurs séparés par un point virgule. La matrice identité 2×2 peut donc se définir comme ceci :

```
I2 = [1 0; 0 1]
```

La notation `beg:end` (ou de manière plus générale `beg:step:end`) génère un vecteur contenant toutes les valeurs de la forme `beg + t*step` pour `t` entre 0 et $\lfloor (\text{end}-\text{beg})/\text{step} \rfloor$.

MATLAB propose toute une série de fonctions préexistantes (e.g. `sin`, `cos`, `exp`, `log`, `abs`, ...) qui peuvent être appelées tant sur des nombres que sur des matrices (et donc des vecteurs) en quel cas l'opération est effectuée individuellement sur chaque entrée de la matrice. Notez que `log(M)` ne calcule pas le log matriciel, il faut utiliser la fonction `logm` pour cela.

Il existe également des fonctions servant à générer des matrices dont `eye` qui génère la matrice identité (ou tout du moins une sous-matrice d'une matrice identité), `zeros` et `ones` qui génèrent respectivement une matrice ne contenant que des 0 ou que des 1, etc. Si un unique paramètre est donné à ces fonctions, alors elles renverront une matrice carrée dont la dimension correspond à ce paramètre. Si par contre plusieurs paramètres entiers sont donnés, alors la *matrice* (ou plus généralement le tableau de matrices) renvoyée aura les dimensions demandées.

Les dimensions d'une matrice peuvent être récupérées via la fonction `size` qui renvoie un vecteur dont les composantes correspondent aux dimensions. La fonction `length` quant à elle renvoie la plus grande des dimensions. La transposée d'une matrice (ou d'un vecteur) se fait via l'apostrophe.

```
>> A = zeros(2, 3)
A =
    0    0    0
    0    0    0
>> size(A)
ans =
     2     3
>> length(A)
```

```
ans = 3
>> A'
ans =
    0    0
    0    0
    0    0
```

Puisque MATLAB est prévu pour manipuler des matrices (et des vecteurs), les opérations usuelles sont celles définies sur les matrices. En particulier, si A et B sont deux matrices carrées, l'instruction $C = A*B$ va prendre le *produit matriciel* entre A et B et si c'est le produit de Hadamard qui est voulu (donc le *produit composante par composante*), il faut utiliser l'instruction $C = A .* B$. Nous pouvons mettre cela en lumière en ayant A la matrice identité $n \times n$ et B la matrice $n \times n$ ne contenant que des 1. La première est l'identité pour le produit matriciel alors que la seconde est l'identité pour le produit de Hadamard. Dès lors $A*B$ devrait donner B alors que $A .* B$ devrait donner A .

```
>> A = eye(2);
>> B = ones(2);
>> isequal(A * B, B)
ans = 1
>> isequal(A .* B, A)
ans = 1
```

De manière plus générale, précéder les opérateurs d'un point permet d'en utiliser la version *component-wise*. Remarquez qu'ici un point-virgule a été placé après les définitions de A et B . Cela permet de ne pas afficher le résultat de l'instruction. De plus, la fonction `isequal` est utilisée ici pour déterminer l'égalité entre plusieurs matrices (ou vecteurs). Cette fonction peut être appelée avec plus de deux paramètres et renvoie 1 si et seulement si toutes les matrices reçues ont les mêmes dimensions et les mêmes entrées aux mêmes indices.

Indicer une matrice (ou un vecteur) se fait avec des parenthèses (contrairement à la plupart des autres langages qui utilisent des crochets), et attention la première position correspond à l'indice 1 (et non 0)! Si $x = [1 \ 2 \ 3]$, alors $x(1)$ est 1 et $x(3)$ est 3. MATLAB permet un indigage un peu plus *fancy* : il est possible d'indicer une matrice avec un tableau afin de choisir les lignes (ou colonnes) à extraire. En particulier, puisque la notation `beg:end` (qui correspond à `beg:1:end`) crée un vecteur contenant toutes les valeurs entières entre `beg` et `end` avec un pas de 1, la notation `x(a:b)` permet donc d'extraire un sous-vecteur de x en ne prenant que les indices entre a et b . Notons que le mot-clef `end` de MATLAB peut également servir lors de l'indigage : `x(2:end-1)` permet par exemple de prendre tous les éléments du vecteur x sauf le premier et le dernier.

Exercice 1.1. Démarrez MATLAB (ou OCTAVE). Dans la fenêtre de commande, tapez `x = -1:0.1:1` et ensuite, exécutez chacune des commandes suivantes :

```
sqrt(x)
sin(x)
x.^3
```

```
plot(x, cos(x .^ 2))
plot(x, sin(x .^ 3))
```

Exécutez les commandes suivantes et expliquez les résultats :

```
x = [2 3 4 5]
y = -1:1:2
x .^ y
x .* y
x ./ y
```

Résolution. Vous pouvez observer ici que `.^` sert bien à mettre chaque élément du vecteur individuellement au cube et que si vous tentez `x ^ 3`, vous aurez une erreur de la part de MATLAB. Le reste des instructions est assez clair : `sqrt(x)` prend la racine carrée de chaque élément, `sin(x)` prend le sinus, etc.

Pour le second point, notez que `y = [-1 0 1 2]` et donc en particulier `y(2) == 0`. Dès lors le deuxième élément de `x ./ y` vaut ici `Inf`.

Exercice 1.2. Initialisez la matrice `A = [1 5 8; 84 81 7; 12 34 71]` et examinez le contenu de :

- `A(1,1)`;
- `A(2,1)`;
- `A(1,2)`;
- `A(3,3)`;
- `A(1:2,:)`;
- `A(:,1)`;
- `A(3,:)`;
- `A(:,2:3)`;
- `A(1:end,:)`.

Que produisent les commandes suivantes :

```
x = 1:1:10
z = rand(10)
y = [z; x]
c = rand(4)
e = [c eye(size(c)); eye(size(c)) ones(size(c))];
d = sqrt(c)
t1 = d*d
t2 = d .* d
```

Résolution. `A(i,j)` donne l'élément à la *i*ème ligne et la *j*ème colonne. Dès lors `A(1,1)` est l'élément dans le coin supérieur gauche, etc. `A(1:2,:)` prend uniquement les lignes dans 1:2 (à savoir les lignes 1 et 2) et toutes les colonnes. `A(1:end,:)` prend tous les éléments de `A`.

Pour la seconde partie, notez que la fonction `rand` renvoie une matrice de nombres pris uniformément dans l'intervalle ouvert $(0, 1)$ dont les dimensions correspondent aux paramètres d'appel. Dès lors `z` est une matrice 10×10 . `y` est quant à lui une matrice de 11 lignes et 10 colonnes puisque `x` est une matrice de dimension 1×10 qui est ajoutée *en-dessous* de `z` pour former `y` (notez bien le point-virgule dans la définition).

`c` est une matrice 4×4 d'éléments pris uniformément dans l'intervalle ouvert $(0, 1)$, donc `size(c)` renvoie un vecteur ligne (donc une matrice 1×2) dont les deux composantes

valent 4. e est donc une matrice définie par bloc et ressemblant à ceci :

$$\begin{bmatrix} c_{11} & c_{12} & c_{13} & c_{14} & 1 & 0 & 0 & 0 \\ c_{21} & c_{22} & c_{23} & c_{24} & 0 & 1 & 0 & 0 \\ c_{31} & c_{32} & c_{33} & c_{34} & 0 & 0 & 1 & 0 \\ c_{41} & c_{42} & c_{43} & c_{44} & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 1 & 1 & 1 & 1 \\ 0 & 1 & 0 & 0 & 1 & 1 & 1 & 1 \\ 0 & 0 & 1 & 0 & 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 & 1 & 1 & 1 \end{bmatrix}$$

Exercice 1.3. Initialisez une matrice A de dimension 4×4 . Étant donné que la fonction `sum` donne la somme des éléments d'un vecteur donné en paramètre, utilisez la fonction `sum` pour trouver la somme des éléments de la première ligne et de la deuxième colonne. Attention à ne compter qu'une seule fois chaque élément !

Résolution. Voici deux solutions possibles :

```
sum(A(:,1)) + sum(A(1,:)) - A(1,1)
sum(A(2:end,1)) + sum(1,:)
```

Exercice 1.4. Soient les trois matrices suivantes :

$$A = \begin{bmatrix} 2 & -3 \\ 4 & 1 \end{bmatrix} \quad B = \begin{bmatrix} 44 & 101 \\ -18 & -41 \end{bmatrix} \quad C = \begin{bmatrix} 1 & 2 \\ -2 & -5 \end{bmatrix}$$

À l'aide de MATLAB, calculez $C^{-1}AC$. Que pouvez-vous en déduire ?

Résolution. Pour déterminer l'inverse d'une matrice, il faut utiliser la fonction `inv` :

```
>> A = [2 -3; 4 1];
>> B = [44 101; -18 -41];
>> C = [1 2; -2 -5];
>> D = inv(C) * A * C
D =
    44    101
   -18   -41
>> isequal(B, D)
ans = 1
```

Donc $B = C^{-1}AC$.

Exercice 1.5. Écrivez un script qui prend en paramètre deux matrices carrées A et B et qui les additionne, les soustrait et les multiplie. Commentez votre script et utilisez la fonction `disp` pour l'affichage.

Résolution.

```
1 function ex5(A, B)
2     disp('Somme :');
```

```

3   C = A+B;
4   disp(C);
5   disp('Différence :');
6   A-B
7   disp('Produit :');
8   A*B
9   end

```

Exercice 1.6. Écrivez un script qui dessine les deux fonctions $f_1(x) = x^2 \cos x$ et $f_2(x) = x^2 \sin x$ sur la même figure. Prenez $x = -2:0.1:2$.

Résolution. Par défaut, chaque nouvel appel à la fonction `plot` efface la figure actuelle et recommence à partir de rien. Pour conserver ce qui a été dessiné sur la figure jusqu'à présent, il faut utiliser l'instruction `hold` :

```

1   x = -2:0.1:2;
2   y1 = x.^2.*cos(x);
3   y2 = x.^2.*sin(x);
4   plot(x, y1);
5   hold;
6   plot(x, y2);

```

Exercice 1.7. L'équation itérative suivante permet de résoudre l'équation $x^2 - x - 1 = 0$:

$$x_{t+1} = 1 + \frac{1}{x_t}$$

pour $t \geq 0$, où $x_0 = 2$. Écrivez un script MATLAB qui implémente ceci à précision ε choisie. Testez votre programme pour $\varepsilon = 5 \times 10^{-4}$.

Résolution.

```

1   function phi = ex7(precision)
2       xprev = 0;
3       xcurr = 2;
4       while abs(xcurr - xprev) >= precision
5           xprev = xcurr;
6           xcurr = 1 + 1./xcurr;
7       end
8       phi = xcurr
9       disp(phi*phi - phi - 1); % devrait être proche de 0
10  end

```

Exercice 1.8. Étant donné A , une matrice de dimension 4×5 , écrivez un script pour trouver la somme de chacune des colonnes en utilisant (i) une boucle `for` et (ii) la fonction `sum`.

Résolution.

```

1 function [colsum1, colsum2] = ex8(A)
2     [rows, cols] = size(A);
3     colsum1 = [];
4     for colidx=1:cols
5         tmp = 0;
6         for rowidx=1:rows
7             tmp = tmp+A(rowidx,colidx);
8         end
9         colsum1 = [colsum1 tmp];
10    end
11    colsum2 = sum(A, 1); % sum(A, 2) for sum of rows

```

Exercice 1.9. Soit x un vecteur de n éléments. Écrivez un script MATLAB pour déterminer le produit

$$P_k = \prod_{i \neq k} x_i = x_1 x_2 \dots x_{k-1} x_{k+1} \dots x_{n-1} x_n.$$

Testez votre script avec des valeurs extrêmes de x et de n .

Résolution.

```

1 function product = ex9(x, k)
2     [rows, cols] = size(x);
3     if(rows > 1)
4         disp('x is not a vector');
5     elseif(k < 1 || k > cols)
6         disp('Incorrect value of k');
7     else
8         product = 1.;
9         for i=1:cols
10            if(i != k)
11                product = product * x(1,i);
12            end
13        end
14    end

```

Notons que nous pouvons également utiliser la fonction `prod` de MATLAB qui, de manière similaire à `sum`, retourne le produit du vecteur passé en paramètre afin d'avoir le produit de toutes les entrées. Or nous savons que $x_k P_k = \prod_{i=1}^n x_i$, donc nous pouvons simplement diviser par x_k . Il faut tout de même que $x_k \neq 0$ pour cela :

```

1 function product = ex9v2(x, k)
2     if(x(1,k) != 0)
3         product = prod(x) / x(1,k);
4     else
5         product = ex9(x, k);
6     end

```

Exercice 1.10. Écrivez un script qui génère une matrice dont les éléments le long de la diagonale valent d et les coefficients directement au-dessus et en-dessous de cette diagonale valent c . Le reste des entrées doit être égal à 0.

Résolution. Utilisons la fonction `diag` qui fonctionne de la manière suivante :

- `diag(A)` pour une matrice A renvoie un vecteur (colonne) contenant tous les éléments de la diagonale;
- `diag(v)` pour un vecteur v renvoie une matrice ayant le vecteur v pour diagonale et des 0 partout ailleurs (en particulier `diag(diag(v))` est égal à v);
- `diag(v, k)` pour un vecteur v et un entier k renvoie une matrice ayant v pour k ème diagonale (à savoir les entrées (i, j) pour lesquelles $j-i$ vaut k) et avec des 0 partout ailleurs.

```

1 function A = ex10(n, a, b)
2     subdiag = ones(n-1, 1);
3     upper_diag = diag(subdiag, 1);
4     lower_diag = diag(subdiag, -1);
5     around_diags = lower_diag + upper_diag;
6     main_diag = diag(ones(n, 1));
7     A = a * main_diag + b * around_diags;

```

Une version plus explicite peut s'écrire comme ceci :

```

1 function A = ex10_explicit(n, a, b)
2     A = zeros(n);
3     for i=1:n
4         for j=1:n
5             if(i == j)
6                 A(i,j) = a;
7             elseif(abs(i-j) == 1)
8                 A(i,j) = b;
9             end
10        end
11    end

```

Exercice 1.11. Écrivez une fonction MATLAB pour résoudre l'équation suivante :

$$ax^2 + bx + c = 0.$$

La fonction doit prendre en entrée les paramètres a , b et c et renvoyer les valeurs des deux racines

Résolution.

```
1 function [x1, x2] = ex11(a, b, c)
2   if(a == 0)
3     disp('a cannot be 0');
4   else
5     D = b*b - 4*a*c;
6     x1 = (-b + sqrt(D)) / (2*a);
7     x2 = (-b - sqrt(D)) / (2*a);
8   end
```

Exercice 1.12. En sachant que la série suivante :

$$\sum_{k \geq 0} A^k = I + A + A^2 + A^3 + \dots$$

converge vers $(I - A)^{-1}$ si les valeurs propres de la matrice A sont bornées par 1 en valeur absolue. La matrice suivante (de dimension $n \times n$) satisfait cette condition si $a + 2b < 1$:

$$\begin{bmatrix} a & b & 0 & \dots & 0 & 0 & 0 \\ b & a & b & \dots & 0 & 0 & 0 \\ 0 & b & a & \dots & 0 & 0 & 0 \\ \vdots & & & \ddots & & & \vdots \\ 0 & 0 & 0 & \dots & a & b & 0 \\ 0 & 0 & 0 & \dots & b & a & b \\ 0 & 0 & 0 & \dots & 0 & b & a \end{bmatrix}$$

Expérimentez avec cette matrice pour différentes valeurs de a , b et n pour illustrer que cette série converge sous la condition annoncée.

Résolution.

```
1 function inverse = ex12(n, a, b)
2   A = ex10(n, a, b);
3   Apow = A;
4   inverse = eye(n);
5   for k=1:100
6     inverse = inverse + Apow;
7     new_Apow = Apow*A;
8     disp('||A^k - A^(k-1)||_inf:');
9     disp(max(abs(new_Apow-Apow)(:)));
10    Apow = new_Apow;
11  end
12  disp('||(sol * (I-A)) - I||_inf:');
13  disp(max(abs(inverse * (eye(n)-A) - eye(n))(:)));
```
